

AI Ethics - Team 15

# (Progress)

## SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models

Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. arXiv.

Youngmin Im (20253647)

Wonseok Jang (20254531)

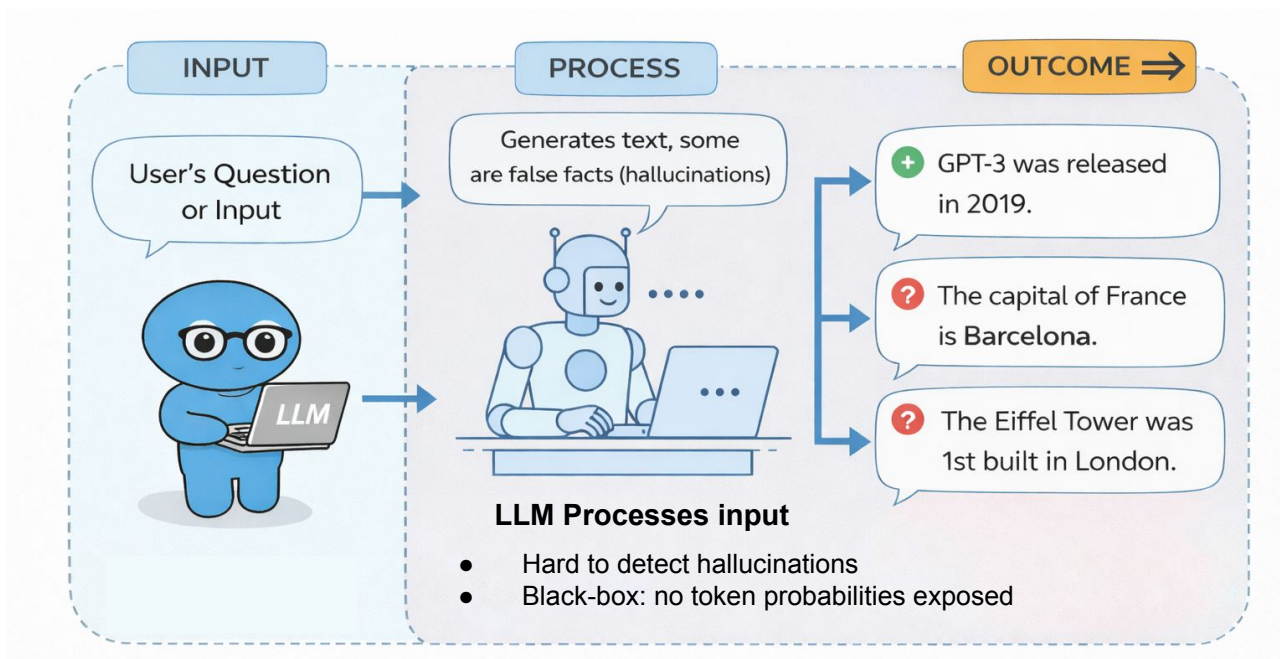
Soyeon Park (20230280)

Mariana Cuevas (20254746)



## Problem

# Hallucination detection challenge for LLMs



⚠ LLMs make hallucinations all the time but there's no convenient method to detect them sentence by sentence

# Replication approach

## SelfCheck GPT

Zero-resource Black-box hallucination detection

### Core Idea

**Factual** → Sampled responses are consistent

**Hallucinated** → Sampled responses diverged

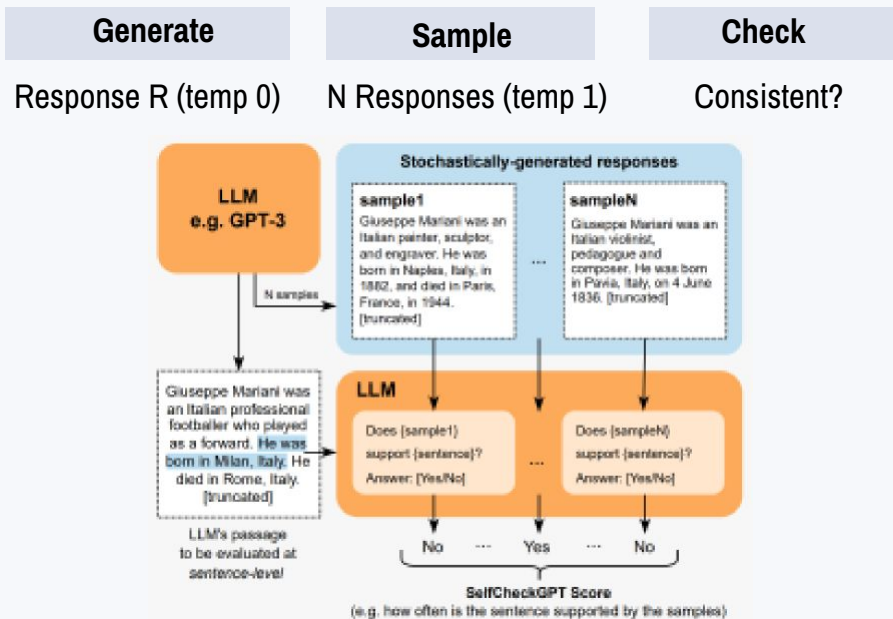
- Black-box: No internal access needed
- Zero-resource: No external DB required
- Outperforms grey-box baselines

### Key Insight

Known facts yield consistent responses;

Hallucinated facts diverge — no external DB needed.

### How it works



# Confidence Prediction

## SelfCheck GPT

Zero-resource Black-box hallucination detection

How it works

How many women Nobel Laureates have existed throughout the years?

Approximate accuracy: 80%

**About 65.** The Nobel Prize and the Sveriges Riksbank Prize in Economic Sciences have been awarded to women 65 times **between 1901 and 2024.**



Ask me anything...



# Confidence Prediction

## SelfCheck GPT

Zero-resource Black-box hallucination detection

How it works

### BERTScore

(Semantic similarity of two texts)

- Black-box: No internal access needed
- Zero-resource: No external DB required
- Outperforms grey-box baselines

### Key Insight

Known facts yield consistent responses;

Hallucinated facts diverge — no external DB needed.

### LLM Prompting

(Ask another LLM if two texts align)

Generate

Response

Sample

Check

### NLI Score

(Logical relationship of two texts)

e.g. GPT-3

N samples

Giuseppe Mariani was an Italian painter, sculptor, and engraver. He was born in Naples, Italy, in 1862, and died in Paris, France, in 1944. [truncated]

Giuseppe Mariani was an Italian violinist, pedagogue and composer. He was born in Pavia, Italy, on 4 June 1836. [truncated]

Does [sample1] support [sentence]? Answer: [Yes/No]

Does [sampleN] support [sentence]? Answer: [Yes/No]

No ... Yes ... No

SelfCheckGPT Score

(e.g. how often is the sentence supported by the samples)

## Replication Approach

### Task division: Team 1 & Team 2

#### Grey LLM Team — Passages 1-119

- Reuse original main passage
- Resample with **GPT-3.5 Turbo**, N=10
  - Observe Log Probs
- Run **NLI + BERTScore**
  - **NLI**: DeBERTa-v2
  - **BERT**: RoBERTa

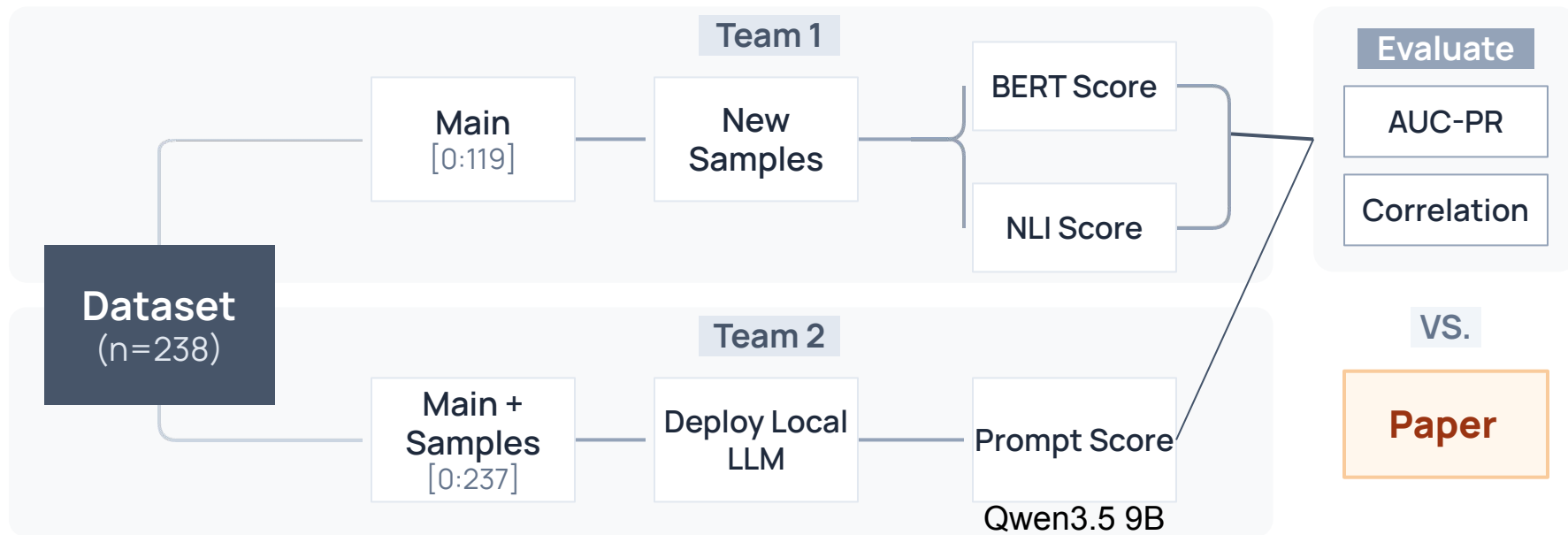
#### Local LLM Team — Passages 1-238

- Run **Prompt variant**
  - 1908 Sentences x 20 Samples  
= **38160 API Calls**
- Test multiple local LLMs
  - Qwen3.5 9B
  - Qwen3.5 9B Q8
  - Gemma4 8B Q4

Both teams share the original human annotations as ground truth for evaluation.

# Replication Approach

## Tasks Workflow



## Replication Issues

### Issue: API & Compute budget

#### ! The Problem

BERT / NLI scoring on Colab CPU

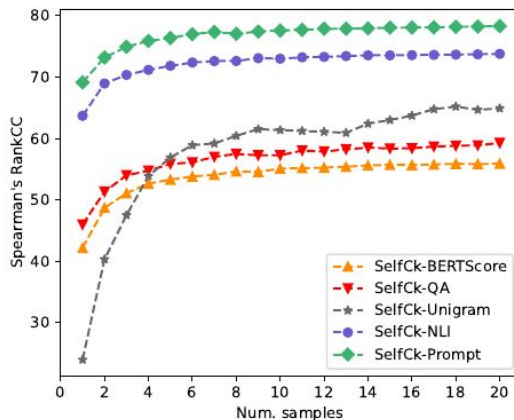
=> **slow & unstable**

Generating **N = 20 samples** x 238 passages

=> **expensive**

#### ✓ What We Did

- Reduced **N: 20 → 10**

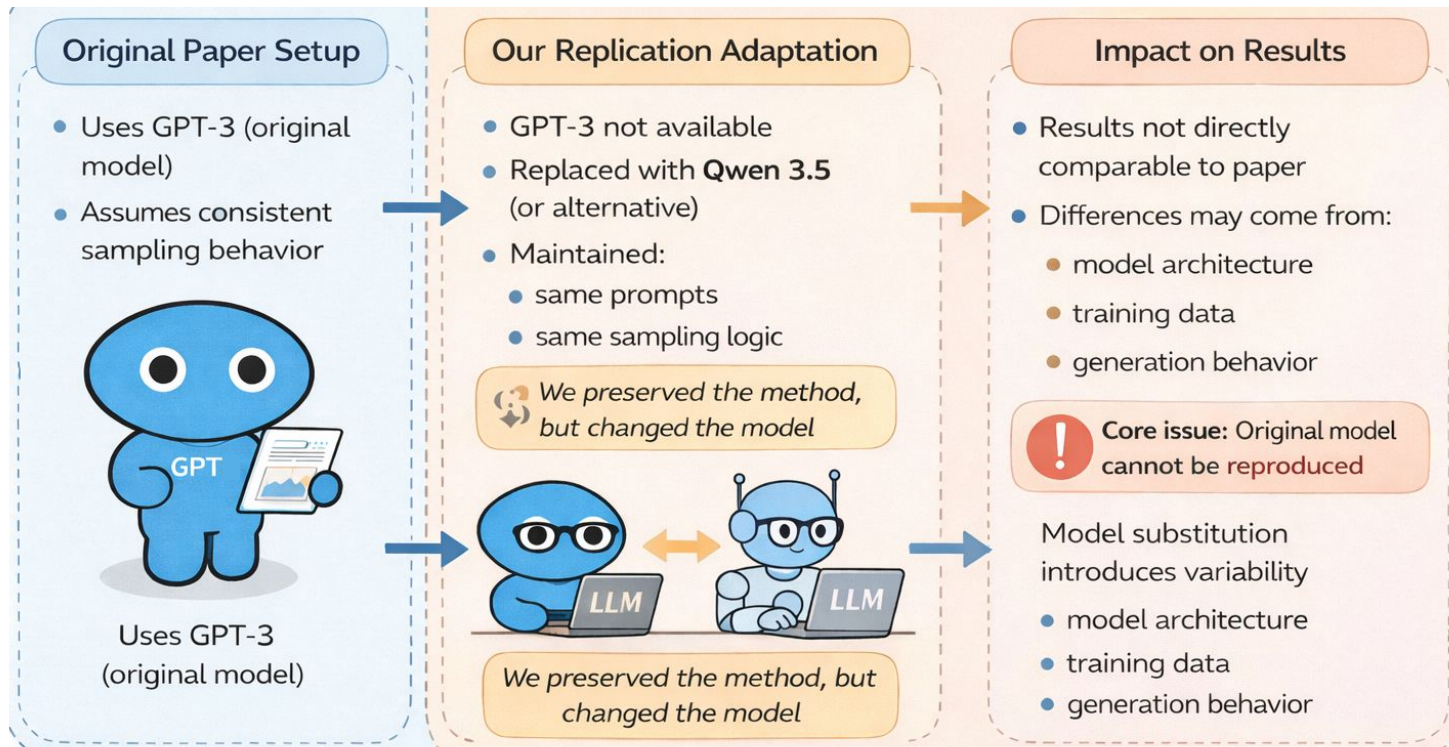


Reduce sample size  
to balance cost and  
feasibility



## Replication Issues

### Issue: Model Availability & Mismatch



## Replication Result

### AUC-PR Evaluation (Dataset Comparison)

Identical

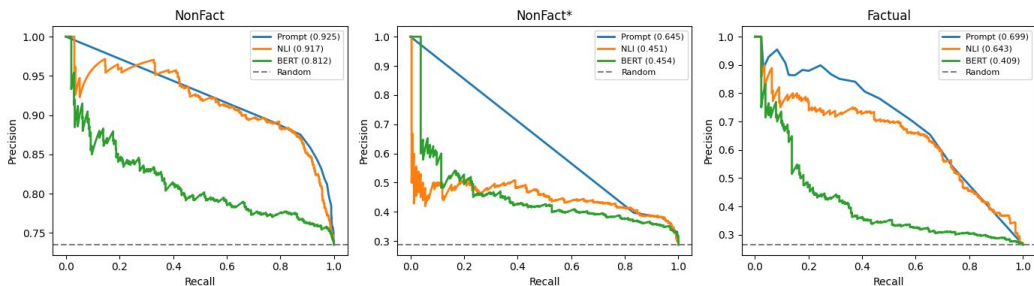
(A sharp drop on BERT's non-factual sentence classification)

| Dataset                      | Method           | NonFact | NonFact* | Factual | Pearson | Spearman |
|------------------------------|------------------|---------|----------|---------|---------|----------|
| GENERATED<br>(GPT 3.5 Turbo) | SelfCk-BERTScore | 83.34   | 37.17    | 45.32   | 56.09   | 55.82    |
|                              | SelfCk-NLI       | 93.99   | 45.55    | 65.44   | 77.20   | 77.92    |
| ORIGINAL<br>(GPT 3)          | SelfCk-BERTScore | 81.76   | 45.64    | 44.06   | 57.92   | 55.30    |
|                              | SelfCk-NLI       | 92.50   | 45.16    | 66.09   | 74.13   | 73.77    |
| PAPER (N=20)                 | SelfCk-BERTScore | 81.96   | 45.96    | 44.23   | 58.18   | 55.90    |
|                              | SelfCk-NLI       | 92.50   | 45.17    | 66.08   | 74.14   | 73.78    |

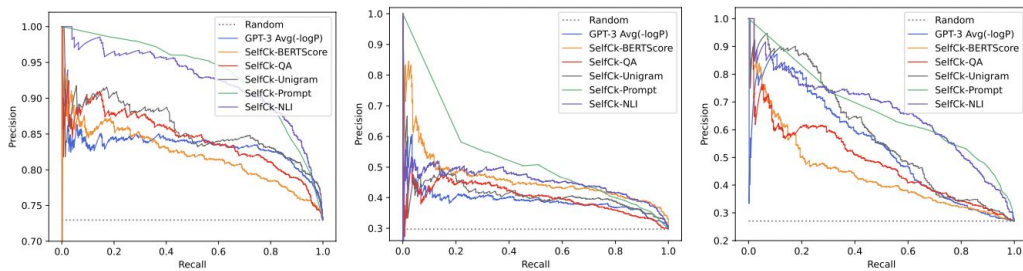
## Replication Result

# Visualization AUC - PR Evaluation

### Replication results



### Reference (SelfCheckGPT Figure 5)



(a) Non-Factual Sentences

(b) Non-Factual\* Sentences

(c) Factual Sentences

Coherent

Replication follows Figure 5 (SelfCheckGPT): same PR-curve setup for NonFact, NonFact\*, and Factual tasks.

## Replication Result

# Better Results on Recent Small Models

| Judge LLM              | NonFact      | NonFact*     | Factual      | Pearson      | Spearman     |
|------------------------|--------------|--------------|--------------|--------------|--------------|
| Qwen3.5-9B Q8          | 93.24        | 65.80        | 74.66        | 75.26        | 76.02        |
| Qwen3.5-9B             | 92.56        | 65.13        | 72.57        | 71.51        | 72.81        |
| Gemma-31B              | 93.15        | 64.51        | 72.40        | 74.06        | 74.60        |
| <i>Paper (ChatGPT)</i> | <b>93.42</b> | <b>53.19</b> | <b>67.09</b> | <b>78.32</b> | <b>78.30</b> |

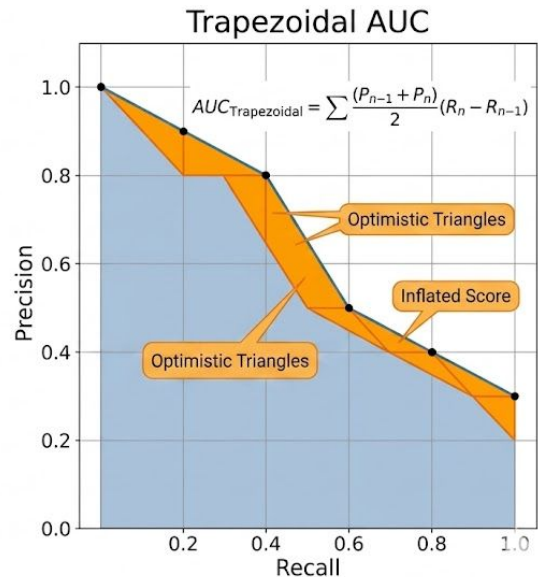
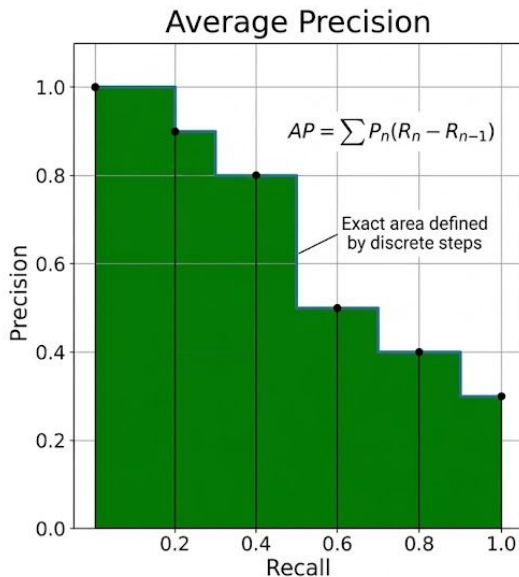
## Key Conclusion

We can now achieve superior performance with **local small models** (sub-9B).  
Once impractical due to cost, is now an efficient reality.

## Replication Issues

### Issue: AUC-PR Evaluation

There are two ways of computing AUC-PR.  
Paper apparently used the Trapezoidal AUC (generally higher)



## Improvement Progress

# Improvement1

Find out and analyze the hallucination that cannot be detected by SelfCheckGPT - that is, the **hallucination with high certainty**

## Dataset

### 1. Sentence log-probs

From the GPT-3.5 Turbo's sample responses  
Sentence logprob = mean token logprob

### 2. Human GT labels

From the paper's sentence-level annotations (Major / Minor / Accurate)

### 3. SelfCheckGPT predictions

Per-sentence factuality scores from our replication runs.

# Improvement1

## Hypothesis1

Among hallucinated sentences, **higher sentence-level log-probability** is associated with **a higher likelihood** of being **misclassified as factual by SelfCheckGPT** (false negative).

## Hypothesis2

High-confidence hallucinations may arise because the model has **internalized incorrect data** during pretraining.

(Cannot directly analyze causality, but provides supporting qualitative evidence)

## Improvement Progress

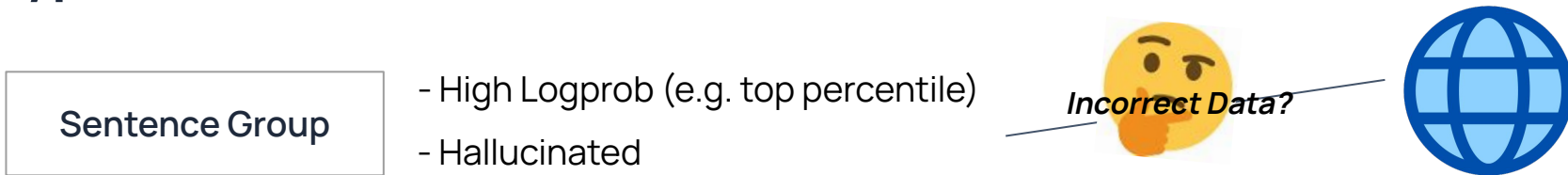
# Improvement1

## Hypothesis1 Method



\* False Negative Rate = (# hallucinated sentences predicted as factual) / (# hallucinated sentences)

## Hypothesis2 Method



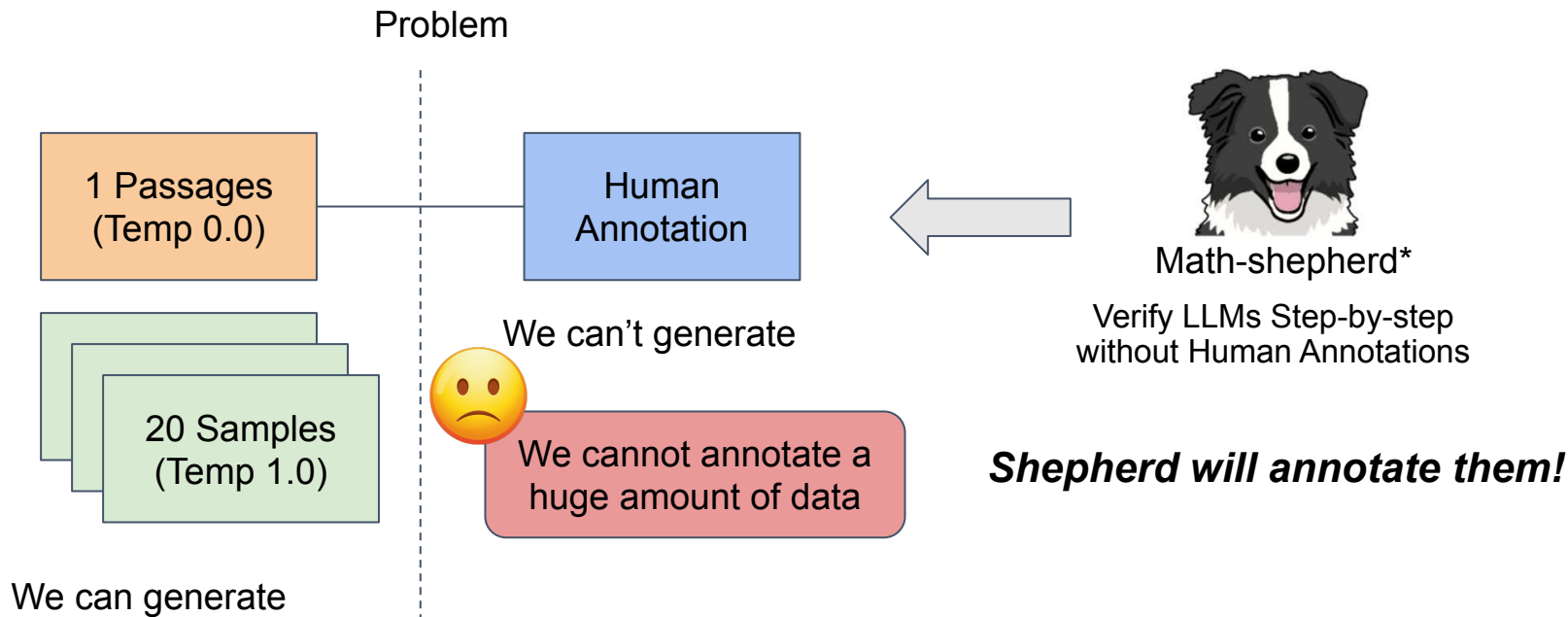


Improvement Progress

**SelfCheckGPT Variants**

**Math & Coding**

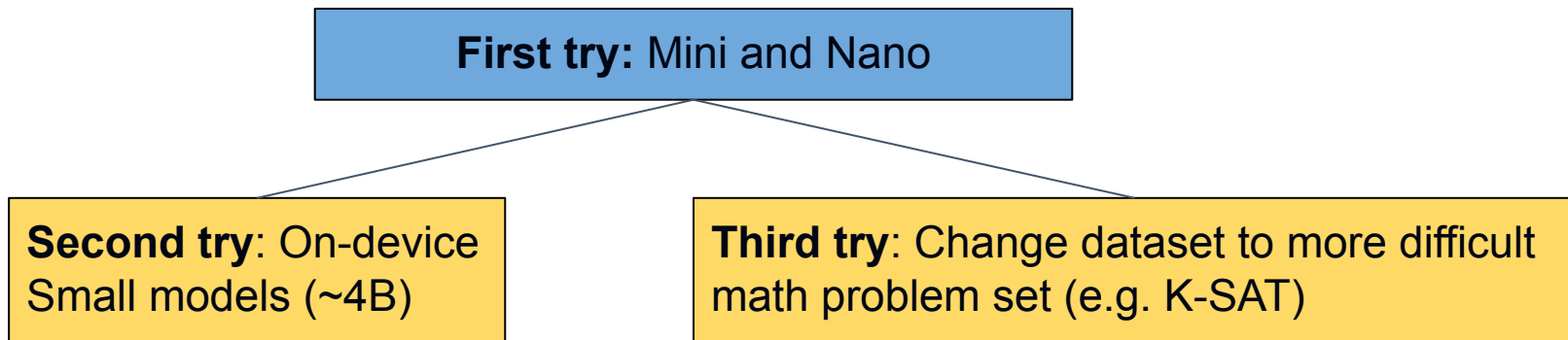
## Extend a Domain to Math



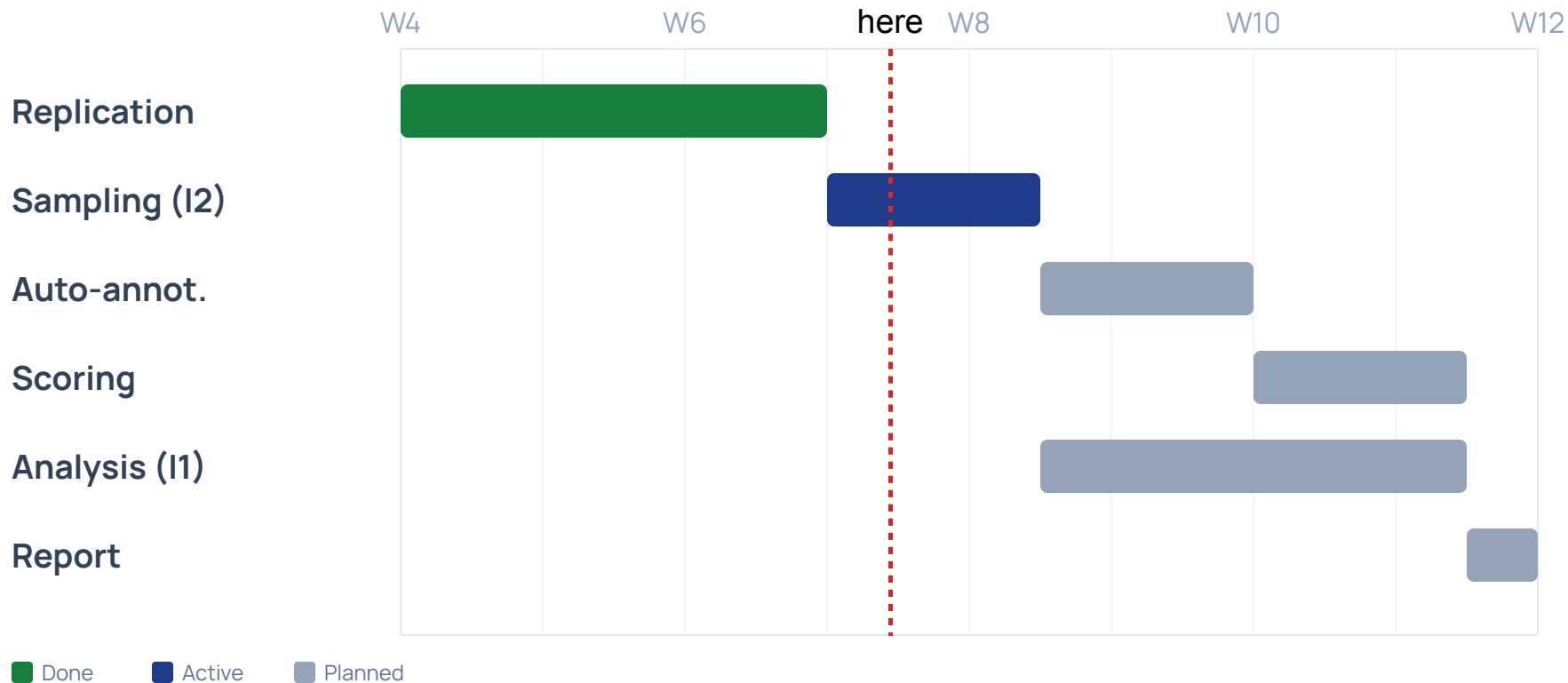
# Recent Models are TOO SMART!

We are going to use 200 problems in GSM8K

GSM8K is about basic arithmetic operations! – Too easy to recent model



# Timeline



# Thank You!

For listening and paying attention

---

**Consistent across samples = factual.  
Inconsistent = hallucinated.**



Math?

**If a model truly understands a solution of  
math problem, it solves it consistently  
across multiple samples**

## Next Step

### 1 Finalize Replication Analysis

Complete the current Sampling (I2) phase and begin the auto-annotation process for the gathered dataset.

### 2 Expand Dataset Benchmarking

Implement SelfCheckGPT variants on GSM8K and HumanEval to test performance on math and coding domains.

### 3 Compile Final Report

Synthesize analysis results and improvement plans into a comprehensive final report by W12.

# Replication Issues

## 1. Computational constraints

### Original paper setup:

- Used 20 sampled generations per prompt
- Applied BERTScore and NLI for consistency evaluation
- Assumes sufficient compute for stable scoring

### Our setup:

- Limited API calls and runtime constraints
- CPU-based execution in Colab
- Reduced samples from **20** → **10**

### Issues:

Consistency-based methods depend on **multiple samples**. So fewer samples would mean weaker estimation of agreement.

**BERT and NLI require high computational cost per comparison** that leads to:

- slower execution
- unstable runs
- incomplete experiments

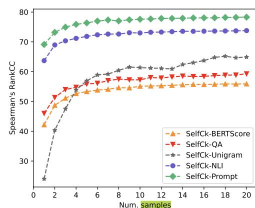


Figure 7: The performance of SelfCheckGPT methods on ranking passages (Spearman's) versus the number of samples.

### Impact on experiment

- Reduced reliability of consistency scores
- Limited ability to compare with original results
- Increased engineering (checkpointing, retries)

### Mitigation

- **Reduce sample size to balance cost and feasibility** (considered the plateau in the paper's graph)
- Use local cheap LLM
- Half and split the number of questions (need more clarification)



## Replication Approach

### Replication: Score

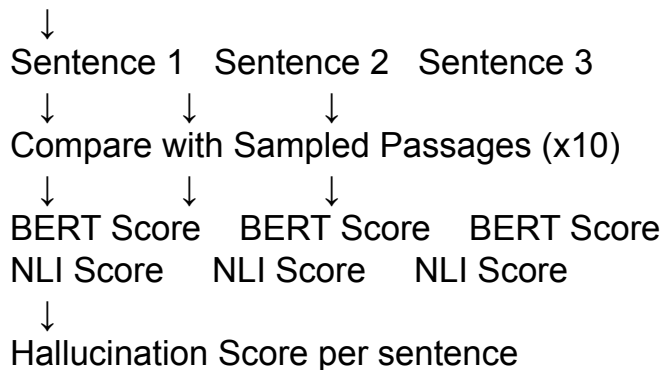
#### Method:

**BERTScore** → measures *semantic consistency* across sampled passages

**NLI (Natural Language Inference)** → measures *logical consistency* (entailment vs contradiction)

#### Ex:

##### Main Passage



Given all relevant resources including private information, LLM agents often **leak sensitive information**



LLM agent is sending email that contains the user's private information (



## Replication Issues

### 3. Slow inference: API Latency and Compute bottlenecks

#### On Causes of slow inference:

- Large number of API calls (~19180) → high waiting time
- API responses limit parallel speed (still behaves sequentially)
- Limited GPU memory → BERT/NLI ran on CPU
- Result: slow and sometimes unstable execution

```
    return sample_i, message

with ThreadPoolExecutor(max_workers=num_samples+10) as executor:
    futures = {executor.submit(_call, i, s): i for i, s in enumerate(sampled_passages)}
    sample_results = [None] * num_samples
    for future in as_completed(futures):
        sample_i, message = future.result()
        sample_results[sample_i] = message
        score = self.text_postprocessing(message)
        scores[sent_i, sample_i] = score
    # print(f" sent {sent_i+1}/{num_sentences} sample {sample_i+1}/{num_samples} →

responses[sent_i].extend(sample_results)
```